JSON

JSON (JavaScript Object Notation) is **a lightweight, text-based data-interchange format that serves as the universal standard for client-server communication and API design in backend engineering**.

Core Role in Backend Engineering

API Data Exchange: Backend servers receive HTTP request payloads and return data to frontends, mobile apps, or third-party services predominantly formatted as JSON.

- **API Data Exchange:** Backend servers receive HTTP request payloads and return data to frontends, mobile apps, or third-party services predominantly formatted as JSON.

Serialization & Deserialization: Backend runtimes convert native language data structures (like structs, classes, or maps) into JSON text strings (**serialization**) for network transport, and parse incoming JSON back into native objects (**deserialization**).

- **Serialization & Deserialization:** Backend runtimes convert native language data structures (like structs, classes, or maps) into JSON text strings (**serialization**) for network transport, and parse incoming JSON back into native objects (**deserialization**).

Language-Agnostic: Although derived from JavaScript syntax, JSON is completely independent of any specific programming language, supported natively or via standard libraries in Node.js, Python, Go, Java, and Rust.

- **Language-Agnostic:** Although derived from JavaScript syntax, JSON is completely independent of any specific programming language, supported natively or via standard libraries in Node.js, Python, Go, Java, and Rust.

Configuration & Storage: Many modern applications use JSON for static/dynamic configuration files or store semi-structured JSON documents directly inside relational and NoSQL databases (e.g., PostgreSQL JSONB, MongoDB).

- **Configuration & Storage:** Many modern applications use JSON for static/dynamic configuration files or store semi-structured JSON documents

directly inside relational and NoSQL databases (e.g., PostgreSQL JSONB, MongoDB).

Key Syntax Rules to Remember

Keys: Must always be enclosed in double quotes (`"key": "value"`).

- **Keys:** Must always be enclosed in double quotes (`"key": "value"`).

Data Types: Supports strings, numbers, booleans (`true` / `false`), `null`, arrays (`[]`), and nested objects (`{}`).

- **Data Types:** Supports strings, numbers, booleans (`true` / `false`), `null`, arrays (`[]`), and nested objects (`{}`).

Restrictions: Does not support comments, undefined values, or trailing commas at the end of lists/object

- **Restrictions:** Does not support comments, undefined values, or trailing commas at the end of lists/object